01 / DAY 1

Understand before changing

A refresher for independent study. Use the short checks on page 6 after reading. This is not an additional assignment.

We inherit a working application for booking spaces. The engineering task is to understand the member promise, change the software safely, and check the result. Writing or generating code is only one part of that work.

Follow the complete delivery cycle

Need > decision > change > evidence
Understand a need; prioritise it; define acceptance; design and implement; review and test; deploy and release; operate; learn and choose the next change.

Example: a member wants to know whether a booking request succeeded. A useful acceptance example states what they should see and what should be stored. “The page loads” is too weak if the booking was not saved.

Trace one member journey

Follow the step	Ask for evidence
Member action	What goal is the person trying to achieve?
Request and code	What input arrives? Which rule or operation handles it?
Stored state	What changes in the database? What must remain unchanged?
Response and observation	What does the member see? Do the response, state and logs agree?
Dependency	Is payment or access real, mocked, or not yet checked?

Words to keep separate

Monolith: an application deployed as one unit; it can contain well-separated modules. **Service:** behaviour exposed to other software through an agreed interface. Separate deployments add communication and failure questions; they are not automatically better.

API means application programming interface: how software asks another component to act or provide information. A **pull request (PR)** proposes a change for review. **Deployment** puts a revision into a running environment; **release** makes a change available for its intended use.

Three different claims
Merged: the change is in the branch. Deployed: that revision is running. Useful: evidence shows that it helps the member. One does not prove the next.

02 / DAY 1

Make a small, checked change

Debugging is the process of choosing evidence that separates possible explanations.

Step	What to do
1. Reproduce	Record the revision, environment, input, expected outcome and actual outcome. State two possible explanations.
2. Discriminate	Choose one observation that would differ between those explanations. Predict the result before checking.
3. Change	Make the smallest relevant change. Keep unrelated restructuring out of the same fix.
4. Verify and review	Retain a regression check, include a boundary case, and ask a peer to repeat the check. Record remaining limits.

Worked debugging example - hypothetical

A booking appears to succeed in the browser, but no booking is visible afterwards. Explanation A: the request was never sent. Explanation B: it reached the server but was rejected. Inspect the browser request and response first: that observation helps distinguish A from B. Then connect the request to stored state and available logs.

An empty log search does not prove that nothing happened. Check the time range, service labels, collection and identifiers. Keep passwords, tokens, card details and access codes out of shared evidence.

What does a passing test establish?

Check	What it can show - and its limit
Rule / unit	One local decision works for the tested inputs; it does not prove the whole journey.
Integration	Collaborating components work together, such as code with a real test database. State which dependencies are simulated.
Contract	An interface keeps the tested promise: inputs, results and meanings. Matching fields alone is insufficient.
End-to-end	The selected journey works through the exercised system. HTTP/database checks alone do not exercise the browser UI.

A **regression test** guards against a problem returning. Derive its expected result from the agreed rule. A **refactor** changes structure while preserving externally observable behaviour; a fix or feature deliberately changes behaviour.

Use AI to learn as well as produce
For unfamiliar work: “Explain the approach, give me one step to try, and help me check my reasoning before automating it.” Verify suggestions against the real system. Be able to explain the decision, the check and what is still uncertain.

03 / DAY 2

Make the business model explicit

Day 1 asks how a request works. Day 2 asks what business promise those steps are meant to keep.

A **domain** is the business activity supported by the software. A **domain model** selects useful concepts, relationships and rules. It helps people predict behaviour and agree responsibilities. It is more than a list of database tables, and it does not require classes.

Separate three kinds of statement

Status	Meaning	Example
Observed implementation	What the inspected code or executed check shows. Name which kind of evidence you have.	An unpaid record currently blocks an overlapping request in an inspected implementation.
Agreed policy	What the business stakeholder says should happen. Record the decision source.	Required payment must complete before confirming a payment-dependent booking.
Proposed design	A suggested way to realise the policy; still open to challenge.	Represent a temporary hold separately from confirmation.

Code reveals implemented decisions, but it does not settle whether those decisions are desirable. If documents and code disagree, ask the stakeholder and retain the unresolved question. A proposed model can also be wrong.

Domain-driven design, in plain language

Domain-driven design (DDD) develops models and language with people who understand the business, then uses those models to guide behaviour and boundaries.

Ubiquitous language is the shared vocabulary used by business participants and developers within a context. A glossary records it; examples reveal where a definition is too broad.

A **bounded context** is a boundary within which terms, models and rules have a consistent meaning. “Booking confirmed”, “payment successful” and “access ready” describe different facts. Qualify the term rather than assuming a universal status.

A **context map** shows relationships between contexts: who establishes a fact or promise, and who relies on it. A bounded context may remain inside a monolith. A table or team name alone does not justify a service boundary.

A useful test of the model
Can two people independently predict the same outcome from the rule? Can they explain who makes the decision and what information that responsibility needs?

04 / DAY 2

Rules become concrete examples

The scenarios on this page are teaching examples. Their stated assumptions are not a report of current deployed behaviour.

1. Valid time is not the same as availability

Assume bookings use intervals that include the start and exclude the end, with no turnover gap. The end must be later than the start. One active booking may occupy a space at a time.

Room A already booked 10:00-11:00	Result under these assumptions
10:30-11:30	Overlaps: reject.
11:00-12:00	Touches the end: passes the overlap rule.
10:00-10:00	Invalid duration: reject.
Room B, 10:30-11:30	No conflict with Room A; still check Room B.

Passing one rule does not prove the complete request is acceptable. Ask separately about permissions, payment and any other agreed conditions. An unpaid hold raises a policy question: does it reserve the interval, and when does it expire?

2. “Paid” may hide two different facts

Suppose a booking is financially covered either by a successful one-off payment or by a subscription. A zero-extra-charge subscription booking can be covered without money being collected for that booking. Define **payment outcome** and **subscription coverage** separately before relying on a flag named “paid”.

For a payment-dependent booking, completing the required payment is a necessary condition for confirmation. It does not by itself establish availability, access readiness or every other condition. Subscription timing remains a policy question: coverage at booking, at the visit, or both?

3. Advertised rate is not agreed amount

Assume a booking keeps its agreed amount. A one-hour booking agreed at 15 currency units remains 15 after the advertised rate rises to 20. The collector needs the agreed amount, with explicit units and currency. Decide the promise before choosing its storage.

4. Capacity and access need precise meanings

For an exclusive room with capacity four, a declared group of four passes a size rule; five does not. A browser that never asks the group size cannot establish the actual group size. Selling individual seats is a different availability model.

Issuance is not entry
Receiving an access credential is different from being allowed through a door at a particular time. A mock code demonstrates simulated issuance, not a working physical lock or enforced access window.

05 / DAY 2

From rules to responsibilities

Start with the information a decision needs. Choose a design that can enforce the rule, then challenge it with examples.

A candidate responsibility map

Context	Decision / meaning	Information crossing a boundary
Purchase	Reservation, availability and agreed booking amount.	Supplies agreed amount and booking reference for collection.
Payments	Payment processing and its outcome.	Reports what happened for the agreed booking and amount.
Integrations/Locks (Access)	Access grant and credential behaviour.	Needs the agreed evidence of eligibility and relevant space/time facts.

This is an illustrative allocation for the course, not proof that three separate deployments are best. Subscription-policy ownership and eligibility coordination still need an explicit agreement. Compare a service split with clear modules inside the same monolith.

Which object should own the overlap rule?

Hypothetical design: a time interval validates start/end and compares intervals. A space schedule knows the active bookings and decides whether a new interval conflicts. A booking tracks its identity and lifecycle. An application operation loads the relevant state, invokes the decision and saves the result.

A new booking cannot determine availability from its own fields alone; it needs information about other bookings. Method names should follow responsibilities, not just grammatical verbs such as “member books space”. Functions can express the same model.

Checking then saving must also remain safe when requests arrive together. Merely naming an operation “reserve”, or wrapping a check and save in a transaction, does not by itself guarantee that concurrent bookings cannot overlap.

Keep a short, useful rule record

Reusable record
Rule ID and statement | scoped terms | ordinary and boundary examples | observed/agreed/proposed status | evidence or decision source | proposed owner and dependency | unresolved question and who can answer it.

The glossary defines terms; the rule record states constraints; the context map explains responsibilities and dependencies. A requirement states a business outcome or rule. A technical specification describes how software is intended to realise it. Link these records rather than keeping conflicting copies.

Bridge to the next session
Trace one rule. Name its owner, inputs, result and side effects. Use these to negotiate a contract before introducing a network call or a separate service.

06 / RETRIEVAL PRACTICE

Check your understanding

Try these without looking back. Then compare your explanation with the answer cues. No submission is required.

1. A PR is merged and CI is green. Has the member problem been solved?

Answer cue: Not established. Identify the deployed revision, exercise the relevant journey and distinguish behaviour evidence from evidence of user value.

2. The logs contain no matching event. Did the booking fail?

Answer cue: Unknown. Check the environment, time range, labels, collection and identifiers; compare the response and stored state.

3. Room A is reserved 10:00-11:00. May another booking start at 11:00?

Answer cue: Under page 4's end-exclusive, no-gap rule, yes for overlap. Other acceptance conditions still need checking.

4. A record says paid, but no money was collected for it. Is it wrong?

Answer cue: Not necessarily: subscription coverage may satisfy the requirement. Establish the field's meaning and the agreed policy before diagnosing it.

5. The database has three tables. Should there be three services?

Answer cue: The table count is insufficient. Consider rules, consistent language, ownership, dependencies and costs; compare modules in one deployment.

6. The model and implementation disagree. Which should you change?

Answer cue: First separate observed behaviour, agreed policy and proposed design. Confirm intended policy with its stakeholder, then choose and verify the smallest justified change.

A two-minute recap with a partner

Explain one member need, one rule, one boundary case and one piece of evidence. Ask your partner for a counterexample. Say what you would check next and what that observation would establish.

How to use this refresher

Prepared from the course topics “Inheriting a Company” (Day 1, 28 September) and “Domain Discovery and Context Mapping” (Day 2, 29 September). Worked scenarios are labelled examples, not a claim that every activity was completed or every application behaviour verified. Use the current shared course glossary and business-rule decisions for project policy.

Keep asking: **What do we know? How do we know it? What is still a decision?**